SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

CSE - Academic Year 2026-27
Java Programming Lab | Batch 2025-29

ASSIGNMENT : 28

Name of Student	Anushree Shinde
PRN No.	25070122041
Batch	CSE A - 2
Class	Java Programming Lab
Academic Year & Semester	2025-29 : Semester 3
Date of Submission	

TITLE : Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Learning Objectives : To implement Remote Method Invocation for distributed Java applications.

Theory : Remote Method Invocation (RMI) is a Java technology that enables an object running on one Java Virtual Machine (JVM) to invoke methods of an object running on another JVM located on a different machine. RMI simplifies distributed application development by allowing remote objects to behave like local objects. It consists of remote interfaces, remote object implementations, the RMI registry, and client-server communication. The registry maintains references to remote objects, allowing clients to locate and invoke them. RMI provides platform-independent communication, object serialization, and automatic network handling, making it suitable for distributed enterprise applications.

Exercise :

1. Create a remote calculCreate a JSP page to display student information entered by the user ator service using Java RMI

```
CalculatorImpl.java
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
public class CalculatorImpl extends UnicastRemoteObject
implements Calculator {
public CalculatorImpl() throws RemoteException {
super();
}
public int add(int a, int b) throws RemoteException {
return a + b;
}
public int subtract(int a, int b) throws RemoteException {
return a - b;
}
public int multiply(int a, int b) throws RemoteException {
return a * b;
}

Calculator.java :
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface Calculator extends Remote {
int add(int a, int b) throws RemoteException;
int subtract(int a, int b) throws RemoteException;
int multiply(int a, int b) throws RemoteException;
int divide(int a, int b) throws RemoteException;
}

1. CalculatorServer.java
import java.rmi.registry.LocateRegistry;

import java.rmi.registry.Registry;
public class CalculatorServer {
```

```
    public static void main(String[] args) {
        try {
            CalculatorImpl calculator = new CalculatorImpl();
            Registry registry = LocateRegistry.createRegistry(2000);
            registry.rebind("CalculatorService", calculator);
            System.out.println("Calculator RMI Server started.");
            System.out.println("Waiting for client...");
        } catch (Exception e) {
            System.out.println("Server Error: " + e.getMessage());
        }
    }
}
```

2. CalculatorClient.java

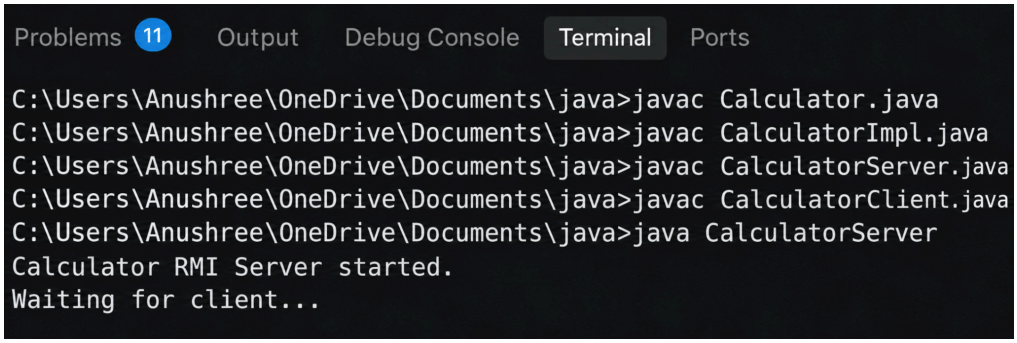
```
import java.rmi.registry.LocateRegistry;

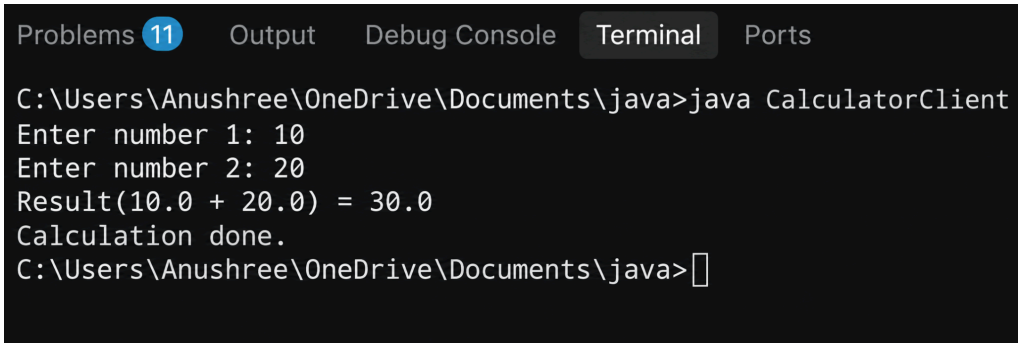
import java.rmi.registry.Registry;
public class CalculatorClient {

    public static void main(String[] args) {
        try {
            Registry registry
                    = LocateRegistry.getRegistry("localhost", 2000);
            Calculator calculator
                    = (Calculator) registry.lookup("CalculatorService");
            int a = 20;
            int b = 10;
            System.out.println("Addition = "
                    + calculator.add(a, b));
            System.out.println("Subtraction = "
                    + calculator.subtract(a, b));
            System.out.println("Multiplication = "
                    + calculator.multiply(a, b));

            System.out.println("Division = "
                    + calculator.divide(a, b));
        } catch (Exception e) {
            System.out.println("Client Error: " + e.getMessage());
        }
    }
}
```

OUTPUT :





2. Create a Java application using **RMI** where the server converts temperature values between **Celsius and Fahrenheit**. The client sends input temperature and receives converted output.

```
1. Temperature.java
import java.rmi.Remote;

import java.rmi.RemoteException;
public interface Temperature extends Remote {

    double celsiusToFahrenheit(double celsius)
            throws RemoteException;

    double fahrenheitToCelsius(double fahrenheit)
            throws RemoteException;
}
2. TemperatureImpl.java
import java.rmi.RemoteException;

import java.rmi.server.UnicastRemoteObject;
```

```
public class TemperatureImpl extends UnicastRemoteObject
        implements Temperature {

    public TemperatureImpl() throws RemoteException {
        super();
    }

    public double celsiusToFahrenheit(double celsius)
            throws RemoteException {
        return (celsius * 9 / 5) + 32;
    }

    public double fahrenheitToCelsius(double fahrenheit)
            throws RemoteException {

        return (fahrenheit - 32) * 5 / 9;
    }
}
```

3. TemperatureServer.java

```
import java.rmi.registry.LocateRegistry;

import java.rmi.registry.Registry;
public class TemperatureServer {

    public static void main(String[] args) {
        try {
            TemperatureImpl temperature
                    = new TemperatureImpl();
            Registry registry
                    = LocateRegistry.createRegistry(1099);
            registry.rebind("TemperatureService", temperature);
            System.out.println("Temperature RMI Server started.");
            System.out.println("Waiting for client...");
        } catch (Exception e) {
            System.out.println("Server Error: " + e.getMessage());
        }
    }
}
```

4. TemperatureClient.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

import java.util.Scanner;
public class TemperatureClient {

    public static void main(String[] args) {
        try {
            Registry registry
                    = LocateRegistry.getRegistry("localhost", 1099);
            Temperature temperature
                    = (Temperature)
registry.lookup("TemperatureService");
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter temperature in Celsius: ");
            double celsius = sc.nextDouble();
            double fahrenheit
                    = temperature.celsiusToFahrenheit(celsius);
            System.out.println(
                    "Temperature in Fahrenheit = "
                    + fahrenheit
            );
            System.out.print("Enter temperature in Fahrenheit: ");
            double fahrenheitInput = sc.nextDouble();
            double celsiusOutput
                    = temperature.fahrenheitToCelsius(fahrenheitInput);
            System.out.println(
                    "Temperature in Celsius = "
                    + celsiusOutput
            );
            sc.close();
        } catch (Exception e) {
            System.out.println("Client Error: " + e.getMessage());
        }
    }
}
```

OUTPUT :

```
C:\Users\Anushree\OneDrive\Documents\java>javac Temperature.java
C:\Users\Anushree\OneDrive\Documents\java>javac TemperatureImpl.java
C:\Users\Anushree\OneDrive\Documents\java>javac TemperatureClient.java
C:\Users\Anushree\OneDrive\Documents\java>javac TemperatureServer.java
C:\Users\Anushree\OneDrive\Documents\java>java TemperatureServer
Temperature RMI Server started.
Waiting for client...
```

```
C:\Users\Anushree\OneDrive\Documents\java>java TemperatureClient
Enter temperature in Celsius: 25
Temperature in Fahrenhett = 77.0
Enter temperature in Fahrenheit: 98.6
Temperature in Celsius = 37.0
C:\Users\Anushree\OneDrive\Documents\java>
```

Outcome :

- Understood the concept of Remote Method Invocation (RMI) in Java.
- Learned the purpose of remote interfaces, remote object implementations, and the RMI registry.
- Understood how a client locates a remote object and invokes its methods.
- Learned to implement remote methods using Remote and UnicastRemoteObject.
- Successfully designed RMI applications for calculator operations and temperature conversion.

Conclusion :

This assignment demonstrated the use of Remote Method Invocation (RMI) for communication between Java applications running in different JVMs. Remote calculator operations and temperature conversion were implemented using remote interfaces, server implementations, and client programs. The assignment helped in understanding remote method calls and the basic structure of distributed Java applications.